

Considerations for Higher Level Transports over Sync

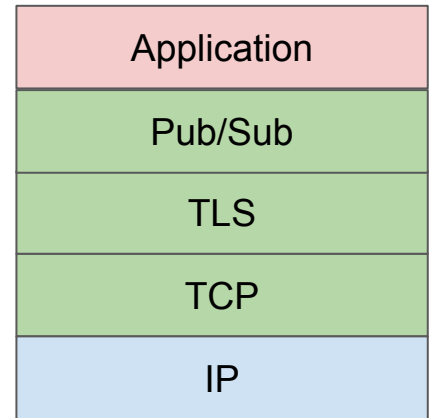
NDN Community Meeting 2021

Varun Patil, Lixia Zhang
UCLA

Randy King, Roger Jungerman
Operant Networks

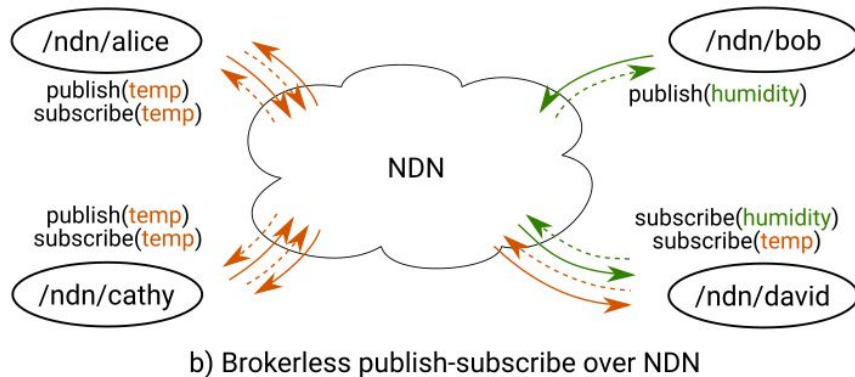
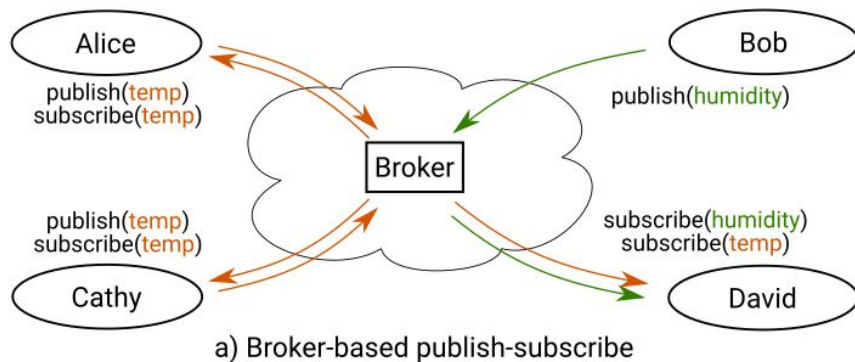
What do applications want?

- Easy to use
 - Dealing with packets is hard
 - Apps produce any number of objects of arbitrary size
 - e.g. TCP over IP
- Built-in security
 - Provide certificates, let the transport do security
 - e.g. TLS over TCP/IP
- Multi-party communication
 - e.g. MQTT over TLS over TCP/IP
- Decentralized



Decentralized Pub/Sub over NDN

- Support a popular design pattern
 - Widely deployed successfully
- Multi-party
- Can be decentralized

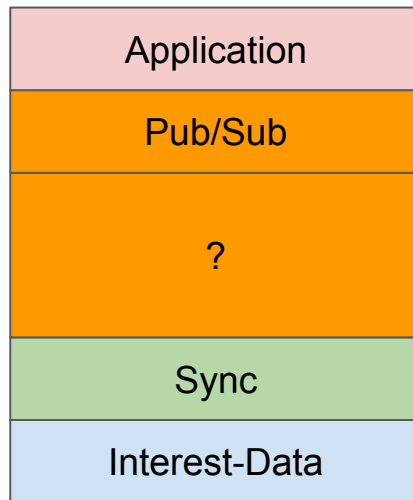


How to support Pub/Sub on NDN?

- Build over Distributed Dataset Synchronization (Sync)
- Sync provides namespace synchronization
- Informs consumer application of new data
 - Transport service of NDN
- Several existing protocols
 - iSync
 - ChronoSync
 - PSync
 - syncps
 - **State Vector Sync**
- Resilient & efficient

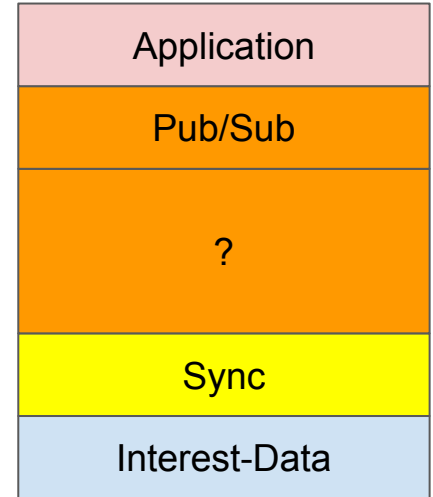
The requirement gap of Sync

- Sync synchronizes namespace for packets
- Developers need higher level APIs
 - e.g. Decentralized Pub/Sub
- Should not need to deal with network layer primitives
 - Interest/Data
 - Per packet signing



Building Pub/Sub over Sync

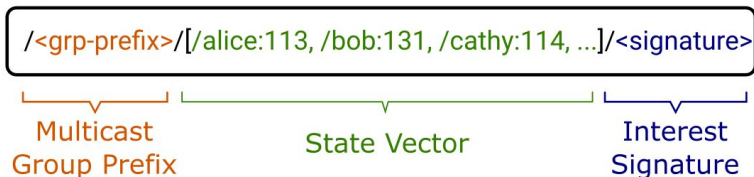
- Sync - namespace synchronisation transport



State Vector Sync (SVS)

- Encodes entire state in a single Sync Interest
 - Loss resilient, since only one Sync Interest needed for synchronization
 - Sent to everyone in group with multicast strategy
- No data reply to multicast Sync Interest
 - Acts purely as update notification to others in the same Sync group
 - Avoid negative impact from potentially multiple replies
- Use of sequence numbers
 - Dataset is list of <producer>:<seq-no> pairs
 - Resilient transport identifier

Sync Interest

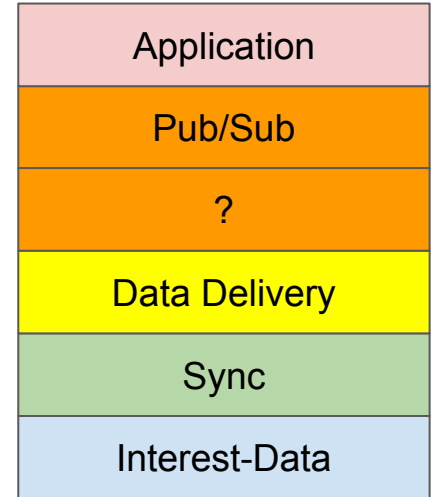


The Application Data Unit

- Named data BLOB
 - Application defined semantics
- Applications do not want to know about network layer
 - No direct creation/manipulation of Interest/Data
 - No MTU limitation
- Can be of any size
 - Higher Level API should handle segmentation
- Sync provides namespace synchronization for packets only
 - Need segmentation for large objects
 - How to fetch data?
 - Sync uses sequence numbers
 - Application uses semantic name

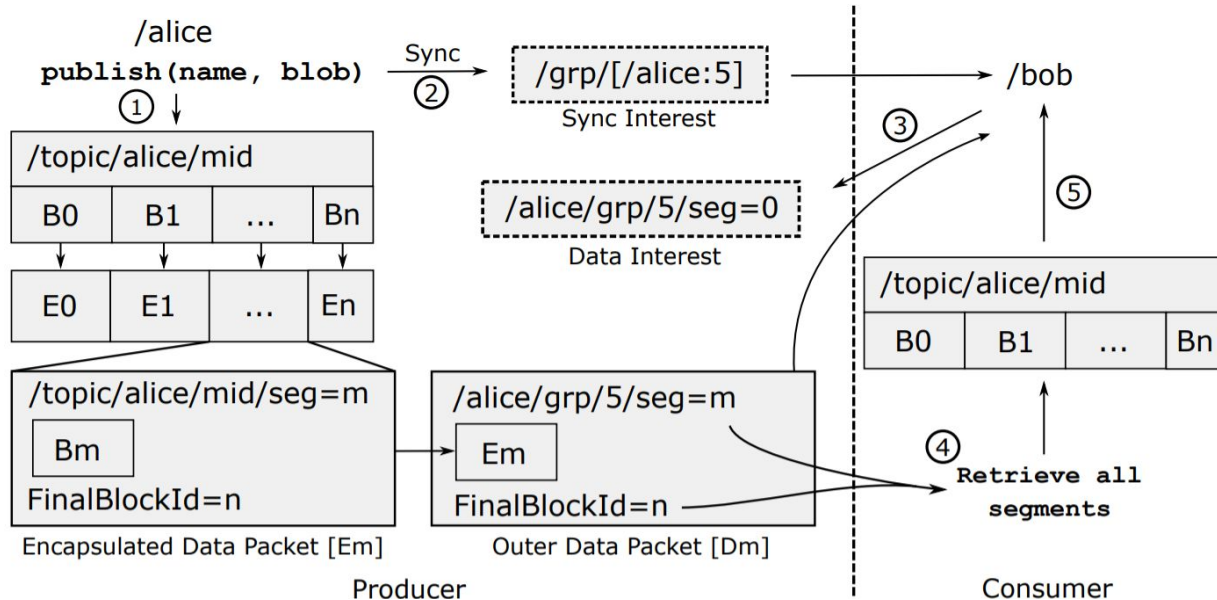
Building Pub/Sub over Sync

- Sync - namespace synchronisation transport
- Data Delivery - use Sync to learn data availability



Data delivery with SVS

- Segment, encapsulate and name using sequence number
- Notify production seq# via Sync
- Fetch

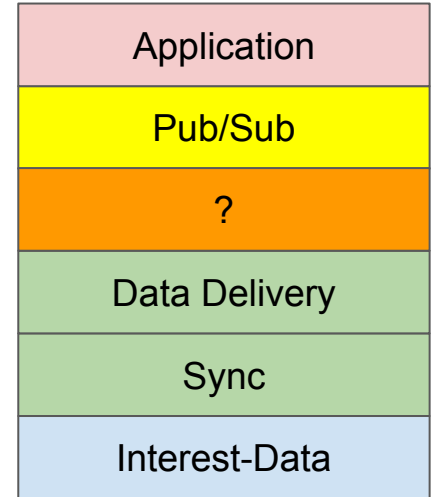


Data Delivery with SVS

- Makes produced data available for everyone in the Sync group
 - Consumer can fetch data and use the contents
- ADUs published can be of arbitrary size
 - Fills the requirement gap partially
- Fetch the data before knowing what it is
 - Due to the use of sequence number - application name not exposed
 - Need a mechanism to inform consumers what is published, so they fetch only what is needed

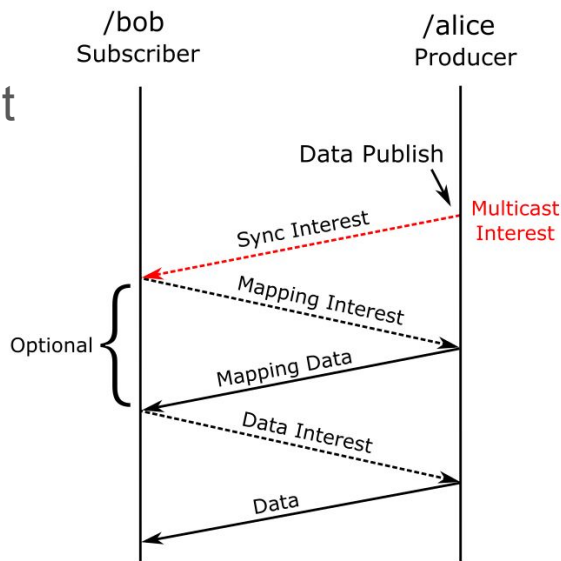
Building Pub/Sub over Sync

- Sync - namespace synchronisation transport
- Data Delivery - use Sync to make data available
- Pub/Sub - fetch only what the application needs



Pub/Sub over SVS

- Application wants to know the data name
- SVS provides only sequence number
- Fetch data name using **Mapping Interest**
- Optimize - can piggyback the name on Sync Interest
 - SVS sends an event-driven Sync Interest on new publication
- Data fetched only for subscribed prefixes (topics)

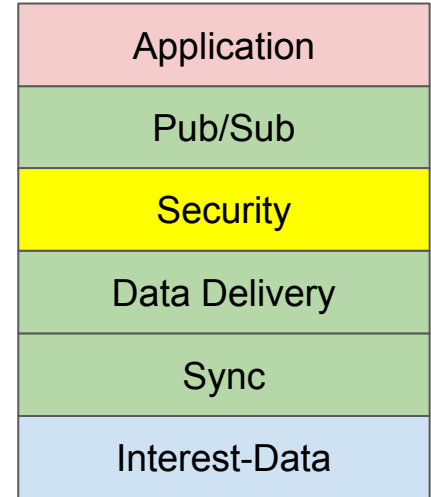


Pub/Sub over SVS

- Applications produce ADUs of arbitrary size
- Only the consumers who need the publication fetch it
- Still need to sign individual packets
 - Need easy-to-use security

Building Pub/Sub over Sync

- Sync - namespace synchronisation transport
- Data Delivery - use Sync to make data available
- Pub/Sub - fetch only what the application needs
- Security

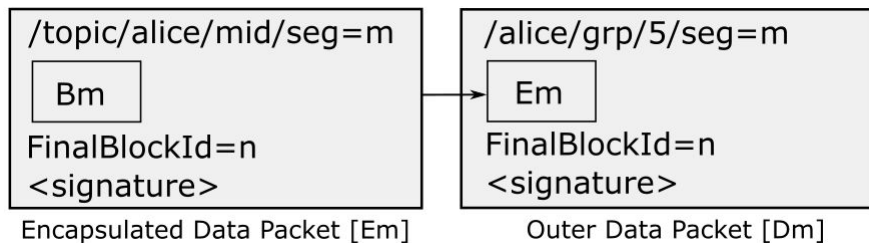


How to do security?

- Who signs?
 - Transport
 - Security Framework
- How to sign?
 - Application provides keys
 - Security Framework may get keys, e.g. for encryption
- What to sign?
 - Application BLOB
 - Transport packets
- How often to sign?
 - Data Aggregation

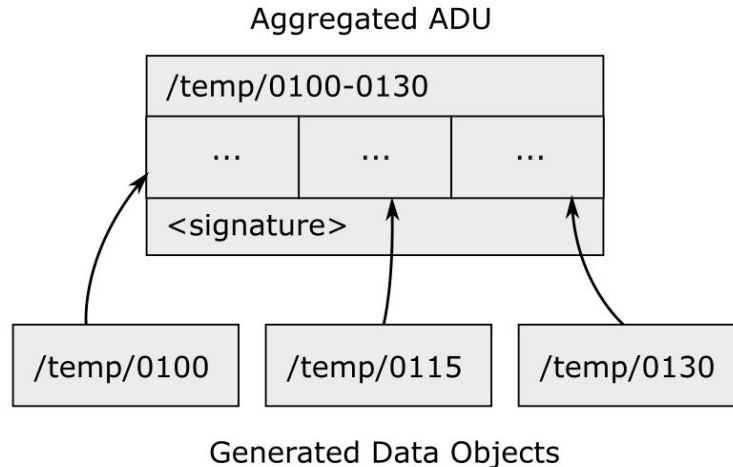
Minimizing Signing Cost

- Packets on the transport may be encapsulated
- Application BLOB must be signed
 - Encapsulated data packet must be signed
 - Reduce cost by aggregation
- What about outer data packet?
 - Asymmetric key signatures
 - High cost for applications producing a large number of ADUs
 - Symmetric key signatures
 - Need a group key distribution mechanism
 - Can encrypt the packet for ADU name privacy



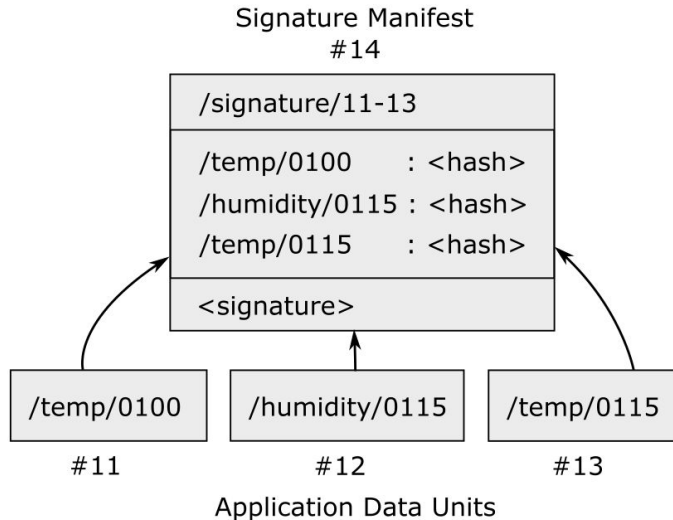
ADU Aggregation

- Application may club multiple ADUs into one before publishing
 - Reduces signing overhead
 - Better network throughput (lower packet count)
 - Semantic naming may be difficult



ADU Aggregation - Signature Manifest

- Alternative approach
 - Publish each ADU immediately on production
 - Periodically publish a signature manifest carrying a signature
 - Manifest contains names and digests of several ADUs
 - Retain semantic naming



ADU Aggregation

- Where to do Aggregation?
 - Pub/Sub library
 - Application defined behavior
- Downsides of Aggregation
 - Adds delay, cannot fetch and use data in realtime anymore

Aggregation Cases - Energy Systems

Case #1: Generation Plant Status Update

- Regularly scheduled readings
- Minimal latency (few 100 ms)
- Guaranteed delivery required
- Large network data bandwidth available



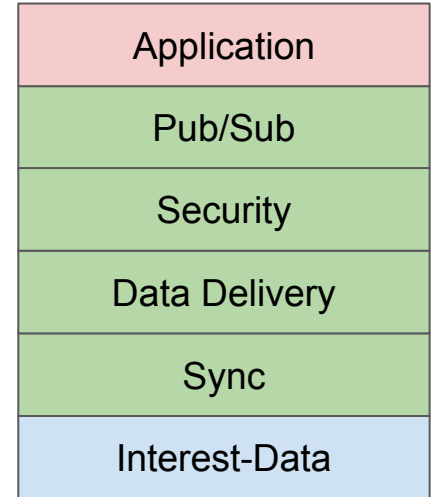
Case #2: Nuclear Power Plant Sensors

- Asynchronous sensor readings
- Hourly updates are sufficient
- Loss of some readings is acceptable
- Limited wireless data bandwidth and capacity



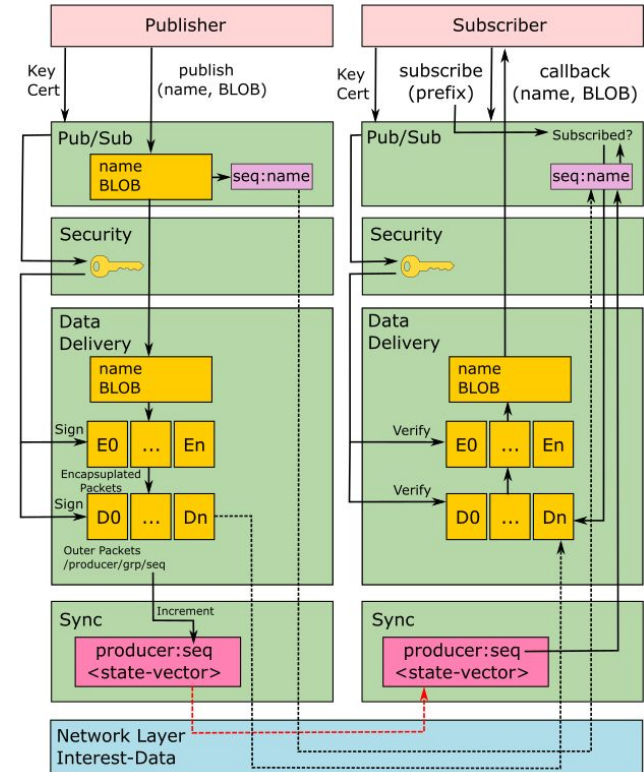
Building Pub/Sub over Sync - Result

- Application publishes named BLOBs (ADUs)
- Automatically signed and/or encrypted
 - Security framework or transport
 - Application provides keys
- Delivered by the transport
 - Only to consumers that want it
 - Regardless of size or number of ADUs
 - Efficiently



Building Pub/Sub over Sync - Result

- Application publishes named BLOBs (ADUs)
- Automatically signed and/or encrypted
 - Security framework or transport
 - Application provides keys
- Delivered by the transport
 - Only to consumers that want it
 - Regardless of size or number of ADUs
 - Efficiently



Future Directions

- SVS scalability
- Investigate data aggregation further

Thank you!